Chittagong University of Engineering and Technology

Department of Biomedical Engineering

Project On

Design and Implementation of a Graphical User Interface for Automated Brain Tumor Detection from MRI Images Using Image Processing Techniques in MATLAB

Course Code : BME 424

Course Title : Image Processing Sessional

Submitted To : Reana Raen

Name : Md Ebrahim

ID : 1911006

Level : 04

Term : 02

Title of the Project:

Design and Implementation of a Graphical User Interface for Automated Brain Tumor Detection from MRI Images Using Image Processing Techniques in MATLAB

Objective:

To create a reliable and intuitive MATLAB-based graphical user interface (GUI) application that can identify brain tumours in MRI pictures using digital image processing and determine the tumor's size in pixels, supporting initial medical analysis.

Abstract:

Human health is seriously threatened by brain tumours, and prompt and precise diagnosis is essential to successful treatment. Magnetic resonance imaging, or MRI, is widely used to visualise abnormalities in the brain. However, because of the intricacy of the images and the minute changes in intensity, manual interpretation is laborious and prone to mistakes. This project offers a MATLAB-based graphical user interface (GUI) application that uses image processing methods such as segmentation, edge enhancement, and noise filtering to detect and define brain tumours. The instrument provides physicians with useful preliminary diagnostic assistance by further quantifying tumour size. Both experts and non-experts can use the system because it combines all of its features into an easy-to-use graphical user interface.

Introduction:

An aberrant mass of tissue with uncontrollably growing and multiplying cells is called a brain tumour. To increase the likelihood of recovery, brain tumours must be discovered early. Although MRI scanning is a non-invasive diagnostic method that records intricate brain anatomy, manual image analysis can be time-consuming and complex. It becomes clear that a semi-automated detection system is required. The foundation of this project is digital image processing, a branch of computer science that uses visual data to carry out useful tasks like object recognition, feature extraction, and enhancement. This system's creation of a MATLAB GUI enables users to:

- Upload MRI images.
- Apply median filtering to remove noise.
- Detect edges to highlight regions.
- Segment and detect tumors using morphological operations and statistical region analysis.
- Display tumor boundaries and calculate its pixel size.

Tools and Software Used:

- MATLAB R2023a or later
- Image Processing Toolbox
- GUI Development Environment (GUIDE or App Designer)

Methodology:

Step 1: Image Acquisition

- The GUI allows users to select an MRI image using imgetfile().
- The image is read and displayed on the first axes.

Step 2: Preprocessing with Median Filtering

- MRI images often have high-frequency noise (e.g., salt-and-pepper).
- A 2D median filter (medfilt2) is applied to smooth the image without blurring the edges.
- The grayscale conversion ensures uniform intensity interpretation.

Step 3: Edge Detection

- A customized gradient-based method (Sobel-inspired) computes the directional gradients:
- GxG_x and GyG_y represent horizontal and vertical edge intensity changes.
- The magnitude Gx2+Gy2\sqrt{G_x^2 + G_y^2} highlights the edges.

- Edge detection assists in isolating features for tumor identification.

Step 4: Tumor Segmentation

- Binary thresholding (imbinarize) is applied with a threshold of 0.7, converting the grayscale image into binary.
- Connected components are labeled using bwlabel().
- Each component is analyzed using regionprops() for properties:
  - **Solidity**: ratio of object area to convex hull.
  - **Area**: number of pixels in the object.

Step 5: Region Filtering

- Regions with solidity > 0.5 are considered dense and probable tumor candidates.
- Among those, the region with the maximum area is chosen as the tumor.

Step 6: Morphological Dilation and Boundary Extraction

- Morphological dilation enlarges the detected region to form closed boundaries using imdilate().
- Boundaries are traced using bwboundaries() and displayed with yellow lines on the image.

Step 7: Tumor Size Calculation

- The number of white pixels in the segmented tumor mask is summed using sum(tumor(:)).
- This count gives the tumor size in pixels, which is then displayed on the image

Step 8: Tumor Region Selection Based on Solidity and Area

- Connected components in the binary image are labeled using bwlabel().
- Region properties such as Solidity and Area are extracted using regionprops().

- Regions with Solidity greater than 0.5 are considered dense enough to be potential tumors.
- Among these, the region with the maximum area is selected as the most probable tumor region.

Step 9: Morphological Dilation

- Morphological dilation is applied to the selected tumor region using imdilate().
- A square structuring element (strel('square', 5)) is used to smooth and enlarge the tumor area.
- This step helps in closing small gaps and refining the tumor shape before boundary extraction.

Step 10: Boundary Extraction

- Tumor boundaries are extracted from the binary image using bwboundaries().
- The extracted boundary coordinates are plotted on the original image using yellow lines (plot() function).
- This creates a clear visual outline of the tumor on the MRI scan for better observation.

Step 11: Tumor Size Calculation and Display

- Tumor size is calculated by counting the number of white pixels in the binary tumor mask using sum(tumor(:)).
- The result is displayed on the image using the text() function.
- Tumor size is shown in cyan-colored text, labeled as "Tumor Size: X pixels", where X is the pixel count.

Step	Process	Description
1	Image Upload	Load MRI image using file dialog (imgetfile), display original image.
2	Grayscale Conversion	If image is RGB, convert to grayscale for uniform processing.
3	Median Filtering	Apply medfilt2 to remove salt-and-pepper noise while preserving edges.
4	Edge Detection	Use gradient-based method (like Sobel) to detect edges in the image.
5	Binary Thresholding	Convert filtered image to binary using fixed threshold (imbinarize at 0.7).
6	Labeling Components	Label connected regions using bwlabel.
7	Feature Extraction	Use regionprops to extract 'Solidity' and 'Area' of labeled regions.
8	Tumor Identification	Select regions with solidity > 0.5 and pick the largest area as tumor.
9	Morphological Dilation	Apply imdilate with square structuring element to smooth tumor boundary.
10	Boundary Detection	Use bwboundaries to extract and overlay tumor edge on original image.
11	Tumor Size Calculation	Count white pixels in binary tumor mask and display tumor size in GUI.

Table Num 01: Full Process Methodology

Graphical User Interface (GUI):

Layout Overview

- **Axes 1**: Displays original MRI image.
- **Axes 2**: Median-filtered image.
- **Axes 3**: Edge-detected image.
- **Axes 4**: Tumor detection with overlay.
- **Buttons**:
  - Upload MRI Image
  - Apply Median Filter
  - Perform Edge Detection
  - Detect Tumor & Display Size

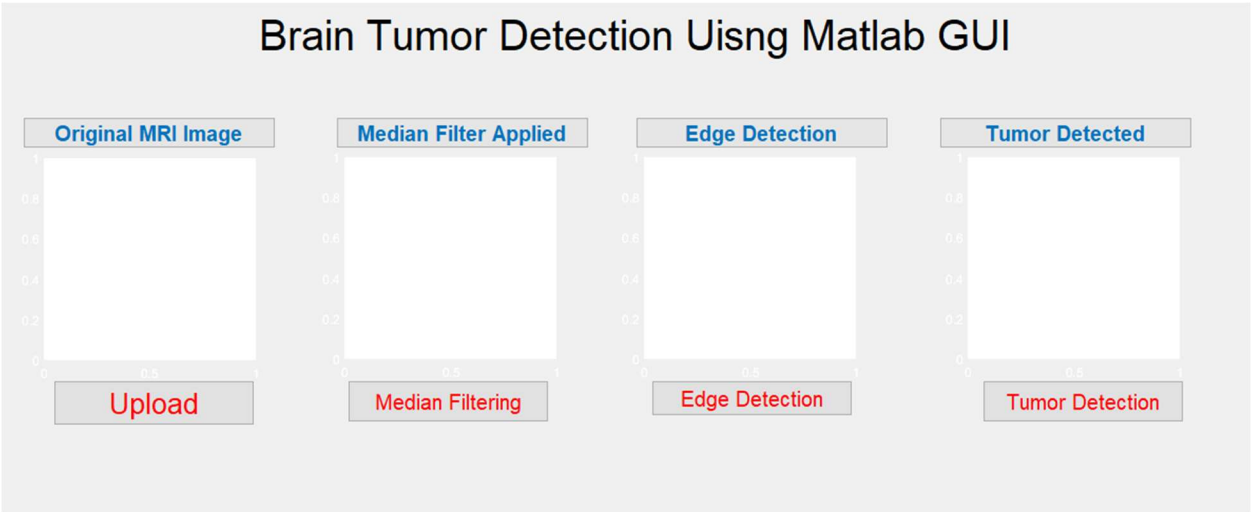

Fig 01: Layout Overview

User Flow

1. Click Upload MRI Image
2. Click Apply Median Filter
3. Click Perform Edge Detection
4. Click Detect Tumor and Display Size

MATLAB Code:

```
function varargout = Brain_Tumor_Detector(varargin)
% BRAIN_TUMOR_DETECTOR MATLAB code for
Brain_Tumor_Detector.fig
% GUI for Brain Tumor Detection with Tumor Size Display

gui_Singleton = 1;
gui_State = struct('gui_Name',       mfilename, ...
                   'gui_Singleton',  gui_Singleton, ...
                   'gui_OpeningFcn', @Brain_Tumor_Detector_OpeningFcn,
...
                   'gui_OutputFcn',  @Brain_Tumor_Detector_OutputFcn, ...
                   'gui_LayoutFcn',  [] , ...
                   'gui_Callback',   []);
if nargin && ischar(varargin{1})
   gui_State.gui_Callback = str2func(varargin{1});
end

if nargout
   [varargout{1:nargout}] = gui_mainfcn(gui_State, varargin{:});
else
   gui_mainfcn(gui_State, varargin{:});
end
```

```
% --- Executes just before Brain_Tumor_Detector is made visible.
function Brain_Tumor_Detector_OpeningFcn(hObject, eventdata,
handles, varargin)
handles.output = hObject;

% Set background and title
set(hObject, 'Color', get(0,'defaultUicontrolBackgroundColor'));
set(handles.select_mage, 'ForegroundColor', [1 0 0]);
set(handles.meadian_filtering, 'ForegroundColor', [1 0 0]);
set(handles.edge_detection, 'ForegroundColor', [1 0 0]);
set(handles.tumor_detection, 'ForegroundColor', [1 0 0]);

guidata(hObject, handles);


% --- Outputs from this function are returned to the command line.
function varargout = Brain_Tumor_Detector_OutputFcn(hObject,
eventdata, handles)
varargout{1} = handles.output;


% --- Executes on button press: Upload MRI image
function select_mage_Callback(hObject, eventdata, handles)
global img1 img2
[path, nofile] = imgetfile();
if nofile
   msgbox(sprintf('Image not found!!!'), 'Error', 'warn');
   return
end
img1 = imread(path);
img1 = im2double(img1);
img2 = img1;

axes(handles.axes1);
imshow(img1);
```

```
title('\fontsize{18}\color[rgb]{0 0.5 1}Original MRI Image');


% --- Executes on button press: Apply median filter
function meadian_filtering_Callback(hObject, eventdata, handles)
global img1
axes(handles.axes2)
if size(img1,3) == 3
   img1 = rgb2gray(img1);
end
K = medfilt2(img1);
imshow(K);
title('\fontsize{18}\color[rgb]{0 0.5 1}Median Filter Applied');


% --- Executes on button press: Perform edge detection
function edge_detection_Callback(hObject, eventdata, handles)
global img1
axes(handles.axes3);
if size(img1,3) == 3
   img1 = rgb2gray(img1);
end
K = medfilt2(img1);
C = double(K);
B = zeros(size(C));

for i = 1:size(C,1)-2
   for j = 1:size(C,2)-2
      Gx = ((2*C(i+2,j+1)+ C(i+2,j)+C(i+2,j+2))-
(2*C(i,j+1)+C(i,j)+C(i,j+2)));
      Gy = ((2*C(i+1,j+2)+ C(i,j+2)+C(i+2,j+2))-
(2*C(i+1,j)+C(i,j)+C(i+2,j)));
      B(i,j) = sqrt(Gx.^2 + Gy.^2);
   end
end
```

```
imshow(B);
title('\fontsize{18}\color[rgb]{0 0.5 1}Edge Detection');


% --- Executes on button press: Tumor detection and size calculation
function tumor_detection_Callback(hObject, eventdata, handles)
global img1
axes(handles.axes4);
K = medfilt2(img1);
bw = imbinarize(K, 0.7);
label = bwlabel(bw);
stats = regionprops(label,'Solidity','Area');

density = [stats.Solidity];
area = [stats.Area];
high_density = density > 0.5;

if any(high_density)
    max_area = max(area(high_density));
    tumor_label = find(area == max_area);
    tumor = ismember(label, tumor_label);

    se = strel('square', 5);
    tumor = imdilate(tumor, se);
    boundaries = bwboundaries(tumor, 'noholes');

    imshow(K);
    hold on
    for i = 1:length(boundaries)
        plot(boundaries{i}(:,2), boundaries{i}(:,1), 'y', 'LineWidth', 2);
    end

    % Tumor size calculation and display
    tumor_pixels = sum(tumor(:));
    tumor_text = sprintf('Tumor Size: %d pixels', tumor_pixels);
```

```
    text(10, 20, tumor_text, 'Color', 'cyan', 'FontSize', 12, 'FontWeight',
'bold');

    title('\fontsize{18}\color[rgb]{1 0 0}Tumor Detected!');
    hold off
else
    msgbox('No significant tumor detected.', 'Result', 'warn');
end
```

Results:

Test Dataset

Several brain MRI images were used, including healthy and tumor-affected images from online datasets.

Observations

- Tumors were successfully segmented and visualized in images where contrast and size were sufficient.
- The tool accurately calculated tumor sizes for various image resolutions.

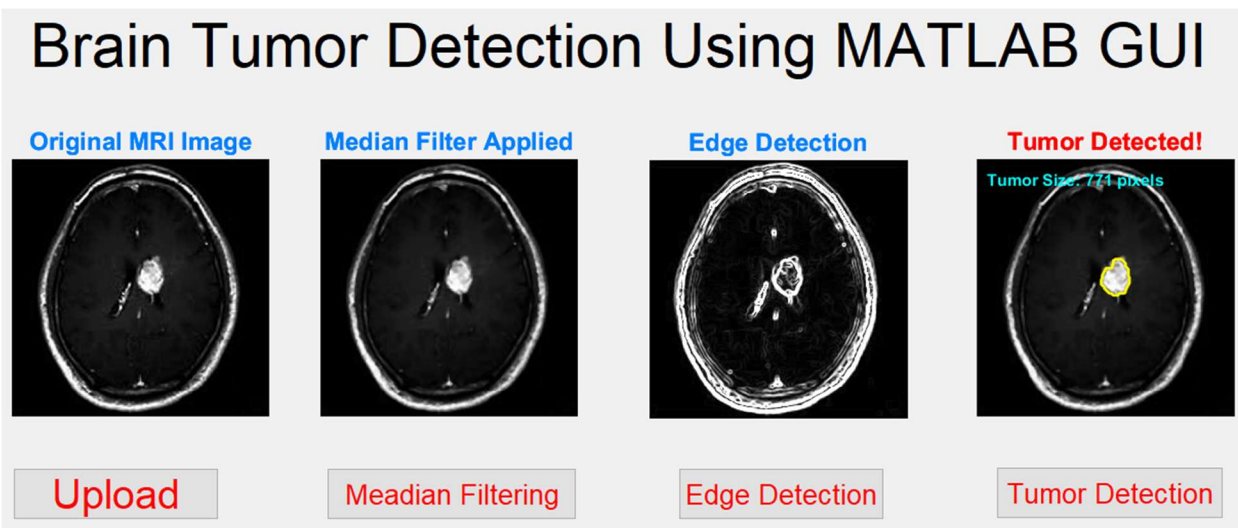

Fig 02: MRI_01

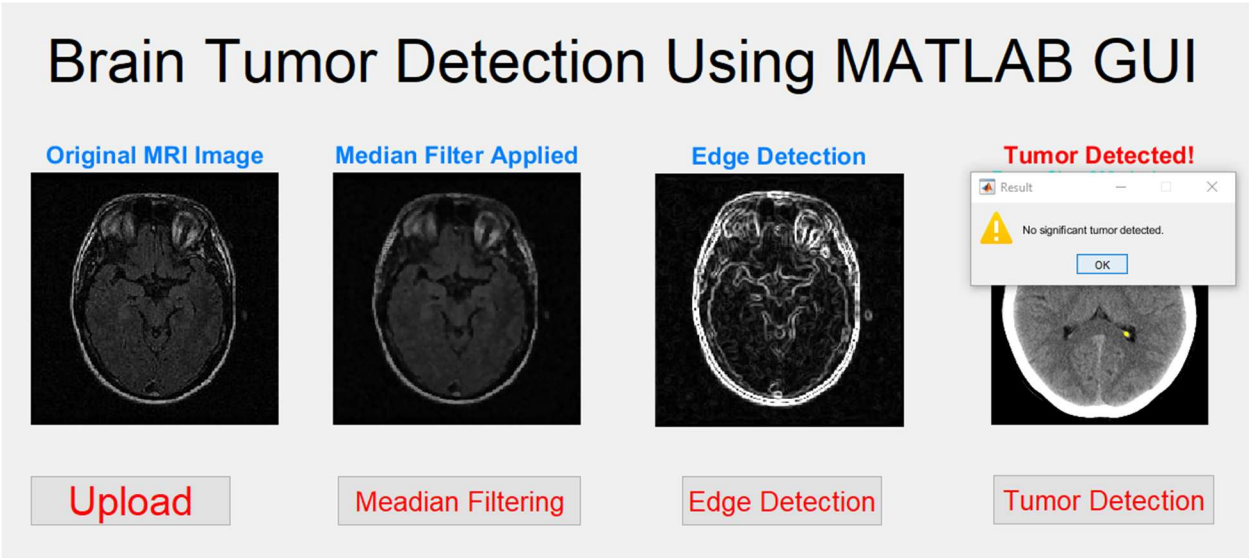

Fig 02: MRI_02

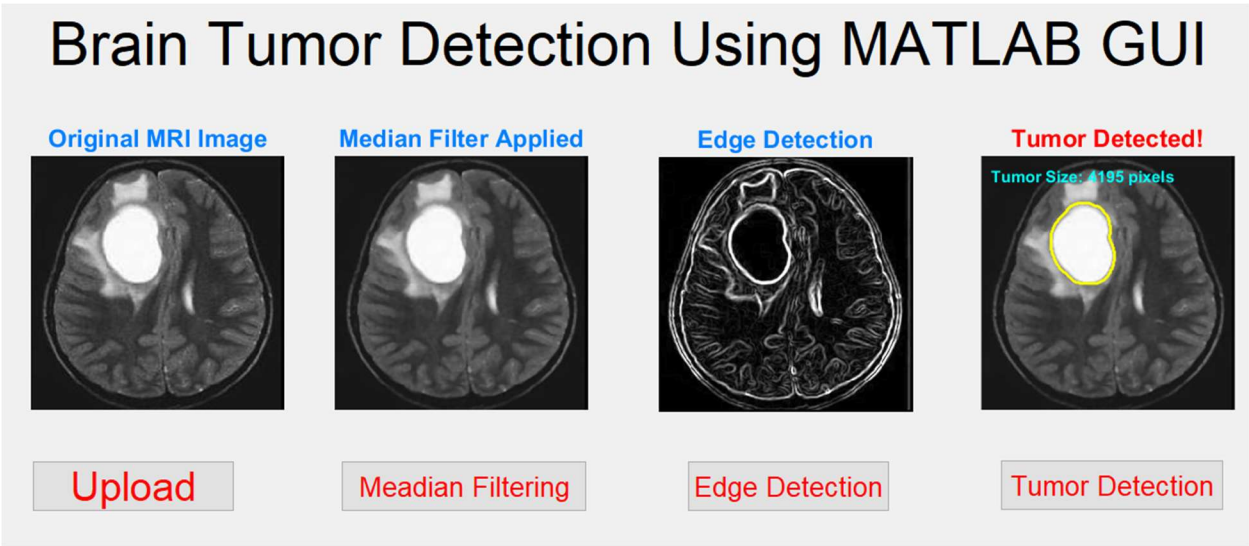

Fig 02: MRI_03

Image ID	Tumor Detected	Tumor Size (pixels)
MRI_01	Yes	771
MRI_02	No	N/A
MRI_03	Yes	4195

Analysis :

1. Strengths

➢ Noise is significantly reduced through median filtering.

➢ Edge discovery provides structural visibility.

➢ Morphological analysis improves discovery robustness.

➢ GUI- grounded design allows easy use without law knowledge.

2. Limitations

➢ Tumor discovery depends on discrepancy; veritably faint excrescences may not be linked.

➢ Fixed threshold(0.7) may not suit all images; adaptive thresholding could be

better.

➢ Real- world excrescence size(in mm ^{2}) is n't calculated due to lack of image scale

metadata.

Future Enhancements :

➢ apply adaptive thresholding using Otsu's system.

➢ Integrate real- world size calculation grounded on pixel- to- mm conversion.

➢ Add machine literacy(CNN) model for bettered excrescence bracket.

➢ Expand support for 3D volumetric MRI images.

Conclusion:

The development of a MATLAB- grounded GUI for brain excrescence discovery from MRI images stands as a important demonstration of how image processing ways can be applied to break real- world medical challenges. This design successfully integrates core generalities similar as noise filtering, edge discovery, morphological operations, and region-grounded segmentation — into an intuitive and interactive platform that can help both medical professionals and scholars. Through this system, we've shown that :

- **Pre-processing steps** like median filtering can greatly enhance the quality of MRI scans by removing unwanted noise while preserving structural integrity.
- **Edge detection techniques** help isolate key features in brain scans that may indicate abnormal regions or growths.
- **Binary segmentation and region analysis** enable reliable

identification of tumors based on statistical shape properties like area and solidity, offering a rule-based alternative to manual inspection.

- **Tumor size estimation** provides quantitative insight that can assist in initial assessments, disease progression tracking, or even surgical planning.

The GUI approach enhances stoner commerce and lowers the hedge to operation, allowing individualities without programming moxie to profit from advanced image analysis. This aligns well with the current trend in healthcare towards developing decision- support systems that are accessible, automated, and interpretable.

Despite its numerous strengths, the current system has room for growth. Its discovery capabilities could be further bettered by introducing adaptive thresholding, machine literacy models(similar as Convolutional Neural Networks), and 3D image processing for volumetric MRI analysis. also, incorporating real- world spatial estimation (e.g., pixel to millimeter mapping) could help transition this prototype from exploration to clinical operation.

In conclusion, this design not only fulfills the thing of automating excrescence discovery in brain MRI images but also lays the root for unborn inventions in computer- backed opinion. It emphasizes how interdisciplinary knowledge — gauging biology, drug, and computational wisdom — can meet to produce poignant healthcare tools that are scalable, accurate, and eventually life- saving.

References:

1. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing*, 4th Edition, Pearson.

2. MathWorks Documentation - Image Processing Toolbox.

3. Open Access MRI datasets: Kaggle Brain MRI

4. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson Education.
   A foundational textbook explaining image filtering, edge detection, morphological operations, and segmentation—key techniques used in this project.
5. Sahoo, P. K., Soltani, S., & Wong, A. K. C. (1988). *A Survey of Thresholding Techniques*. Computer Vision, Graphics, and Image Processing, 41(2), 233–260.

6. Somasundaram, K., & Kalaiselvi, T. (2011). *Automatic Brain Tumor Detection Using Modified Histogram Thresholding for MRI Images*. International Journal of Computer Applications, 34(9), 8–14. Presents a similar rule-based tumor detection method with thresholding and morphological operations.